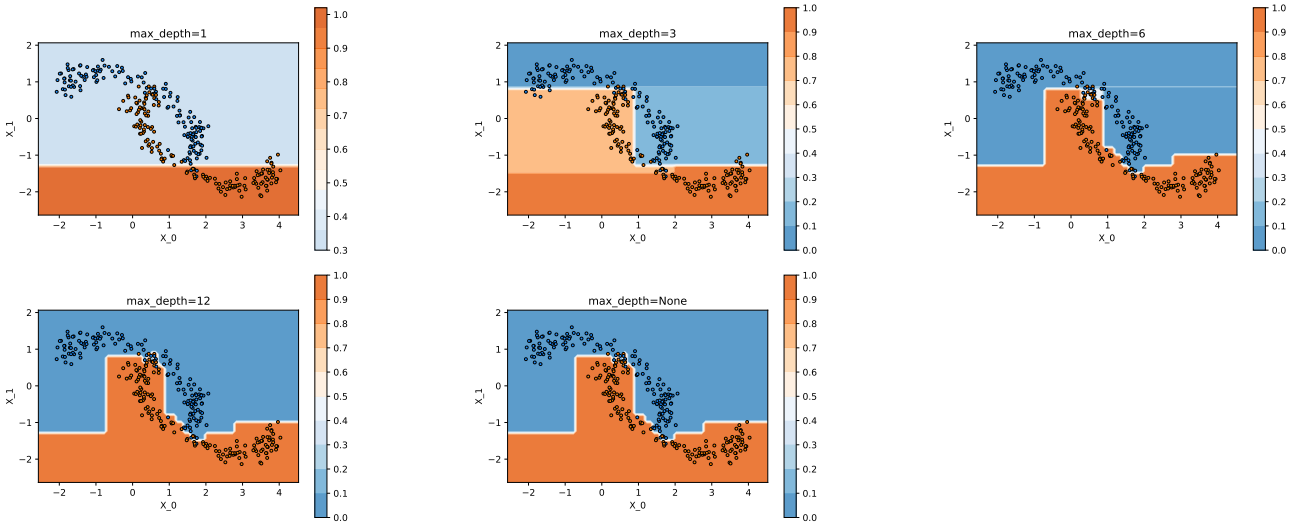
# Project 1 – Classification algorithms

Aurian Borzée (ID), Angélique (ID), Thomas Noël (ID)

# 1 Decision tree

## 1.1 Decision boundaries and model complexity




**Test accuracy, 5 generations**

| `max_depth` | mean ± std |
|---|---|
| 1 | $0.731 \pm 0.015$ |
| 3 | $0.887 \pm 0.019$ |
| 6 | $0.951 \pm 0.010$ |
| 12 | $0.950 \pm 0.020$ |
| None | $0.949 \pm 0.020$ |

Figure 1: Decision tree. Boundaries: seed 0, 1200/300 learning/test points, drawn on the test points (background: $P(y{=}1|x)$). Table: seeds 0–4.

**(a)** A tree only makes axis-aligned splits, so each boundary is a union of rectangles. Depth 1: one split. Depth 3 (7 leaves): a coarse staircase. Depth 6 (28 leaves): the moons are followed. Depth 12 and `None` (54, 56 leaves): small rectangles isolate single points. The moons are oblique, hence the many leaves needed.

**(b)** *Underfitting*: depths 1 and 3, with low accuracy on both sets (train/test 0.748/0.713 and 0.877/0.900), i.e. a bias error. *Overfitting*: depth 12 and `None` reach 0.998 and 1.000 on the learning set but their test accuracy (0.960, 0.957) is no better than depth 6 (0.950); the train–test gap grows from 0.024 (depth 6) to 0.038 and 0.043 as rectangles fit noisy points. Depth 6 is the best compromise.

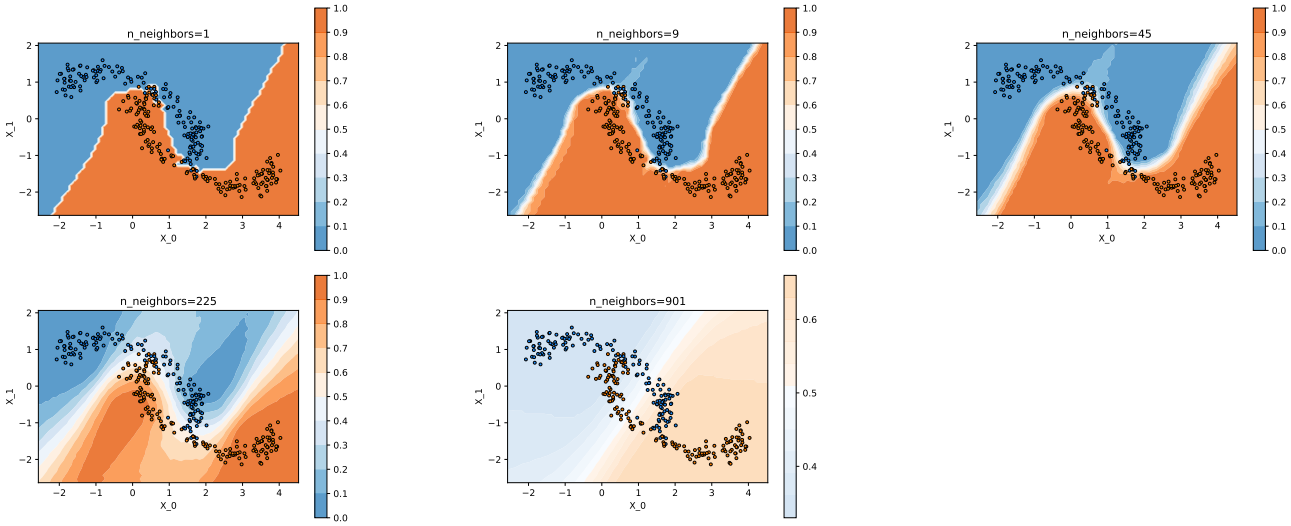
**(c)** Deeper trees have purer leaves: the mean $\max_y P(y|x)$ on the test points is 0.745, 0.872, 0.971, 0.999, 1.000. At maximal depth each leaf holds one class, so `predict_proba` is 0 or 1 almost everywhere: this confidence reflects memorisation of the learning set (variance), not reliability.

## 1.2 Accuracy over five generations

Accuracy rises up to depth 6 (0.731, 0.887, 0.951), then stagnates (0.950, 0.949); the std doubles for depth 12 and `None` (0.020 vs 0.010): deep trees are less stable. Depth 6 has the best mean and the lowest variance.

# 2 K-nearest neighbors

## 2.1 Decision boundaries and number of neighbors




**Test accuracy, 5 generations**

| `n_neighbors` | mean ± std |
|---|---|
| 1 | $0.947 \pm 0.015$ |
| 9 | $0.959 \pm 0.012$ |
| 45 | $0.958 \pm 0.011$ |
| 225 | $0.953 \pm 0.011$ |
| 901 | $0.619 \pm 0.021$ |

Figure 2: K-nearest neighbors: boundaries (seed 0) and accuracy over seeds 0–4.

**(b)** For $k$=1 the boundary is irregular: each learning point owns a region (train 1.000, test 0.953) and noisy points make it jagged (high variance). For $k$=9 and 45 it is smooth and follows the moons (test 0.950, 0.953). For $k$=225 the boundary is very smooth and the probabilities become blurred near the moons (0.947/0.947). For $k$=901 (three quarters of the learning set) each prediction averages most of the data: the boundary is almost a straight line and the probabilities stay in about $[0.35, 0.65]$: underfitting (0.657/0.587).

## 2.2 Accuracy over five generations

Accuracy is almost constant for $k$ from 9 to 225 (0.953–0.959, within one std) and collapses for $k$=901 (0.619). $k$=1 is slightly worse (0.947, std 0.015) because of its variance.

# 3 KDE Naive Bayes

## 3.1 Equivalence of (3) and (1)

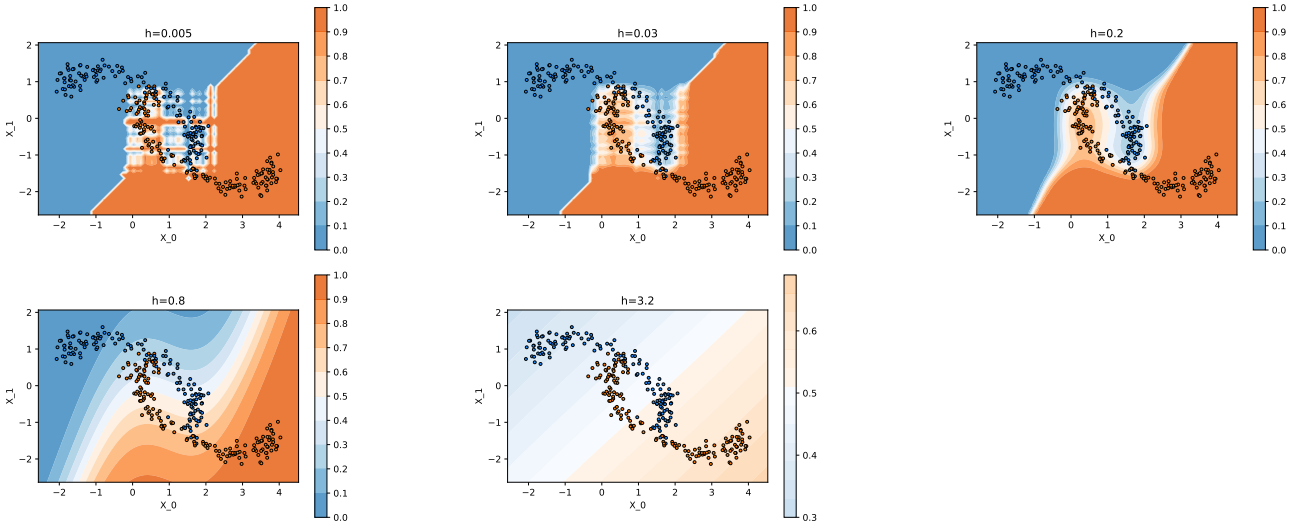
By Bayes' rule $p(y|x) = p(y)\,p(x|y)/p(x)$ and $p(x)$ does not depend on $y$, so $\operatorname{argmax}_y p(y|x) = \operatorname{argmax}_y p(y)\,p(x_1, \ldots, x_p|y)$. By the chain rule $p(x_1, \ldots, x_p|y) = \prod_i p(x_i|y, x_1, \ldots, x_{i-1})$ and by (2), with $Z = \{X_1, \ldots, X_{i-1}\}$, each factor equals $p(x_i|y)$. This gives (3).

## 3.2 Implementation

`fit` stores the sorted classes, the priors (class ratios) and the learning points of each class, which define the 1D Gaussian KDEs (4)–(5). We compute $\log p(y) + \sum_i \log \hat{p}(x_i|y)$, each KDE term with a log-sum-exp over the

learning points (kernels underflow for small $h$), and normalise the posterior with a log-sum-exp over the classes. The result matches scikit-learn's `KernelDensity`.

### 3.3 Effect of the bandwidth




**Test accuracy, 5 generations**

| $h$ | mean ± std |
|---|---|
| 0.005 | $0.838 \pm 0.015$ |
| 0.03 | $0.905 \pm 0.024$ |
| 0.2 | $0.937 \pm 0.016$ |
| 0.8 | $0.760 \pm 0.035$ |
| 3.2 | $0.640 \pm 0.023$ |

Figure 3: KDE Naive Bayes: boundaries (seed 0) and accuracy over seeds 0–4.

For $h$=0.005 each 1D density is a spike on every learning value, so the posterior is an irregular grid-like pattern and the train accuracy (0.952) is far above the test one (0.860): overfitting. For $h$=0.03 the densities are smoother and both agree (0.922/0.920); $h$=0.2 is the best trade-off (0.934/0.943). For $h$=0.8 and 3.2 the densities are over-smoothed and the classes get similar wide marginals: for $h$=0.8 the boundary is a wide smooth band that no longer follows the shape of the moons, and for $h$=3.2 it is almost a straight line with a posterior within about $[0.3, 0.7]$: underfitting (0.769/0.733, then 0.676/0.603).

### 3.4 Accuracy over five generations

The accuracy peaks at $h$=0.2 (0.937), is lower for $h$=0.03 (0.905) and 0.005 (0.838, variance) and drops for large $h$ (0.760, 0.640); the std is largest at the extremes (up to 0.035). Even at its best $h$, NB stays below the tree and KNN (0.95–0.96), see Section 4.

## 4 Method comparison

### 4.1 Tuning protocol

The candidates are the values above. We use a stratified 5-fold cross-validation *on the learning set only*: each value is trained on 4 folds and scored on the held-out one, the best mean accuracy is kept and the model is refitted on the whole learning set; the test set is only used at the end. Each fold trains on 960 points, so $k$=901 remains possible (a 75/25 hold-out leaves 900 points, fewer than $k$).

## 4.2 Accuracy on the two-moon dataset

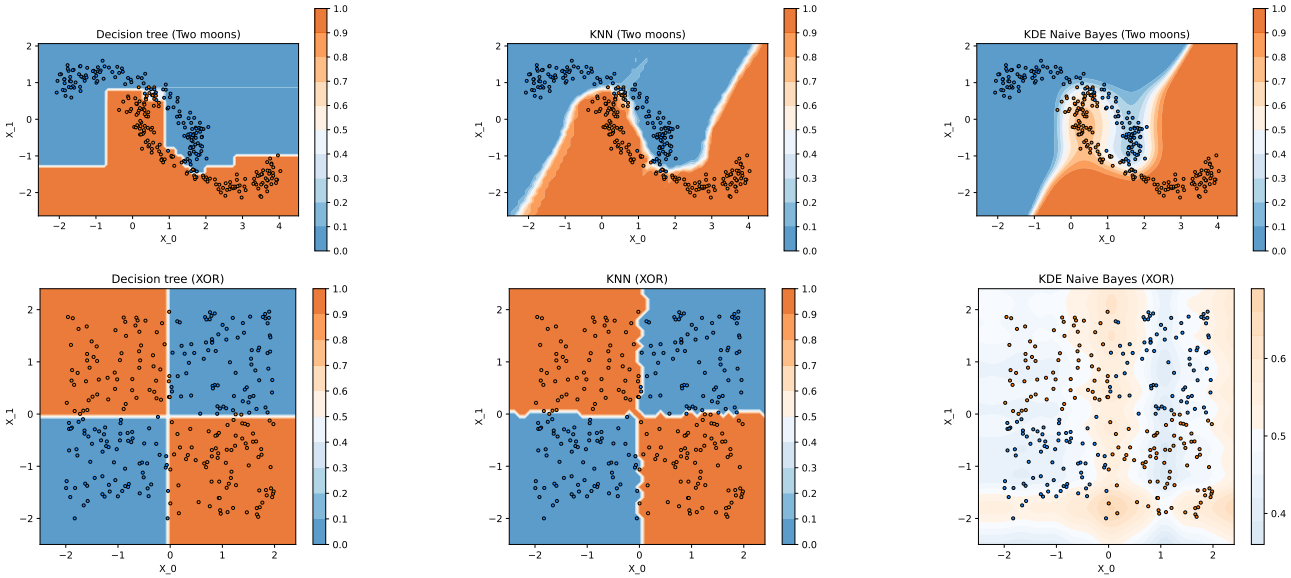



Figure 4: Tuned models (seed 0, test points). Top: two moons. Bottom: XOR.

Mean test accuracy ± std over seeds 0–4: **decision tree** $0.954 \pm 0.013$, **KNN** $0.960 \pm 0.012$, **KDE Naive Bayes** $0.937 \pm 0.016$. Selected values: depth 6 (three times) or 12, $k$=9 (four times) or 45, $h$=0.2 every time.

## 4.3 Ranking on the two moons

KNN $\geq$ tree $>$ KDE Naive Bayes. KNN and the tree differ by less than one std, so five generations cannot separate them; NB is about two points below. KNN and the tree make no assumption on how $X_0$ and $X_1$ depend on each other: KNN follows the moons smoothly, the tree with a staircase. NB assumes that $X_0$ and $X_1$ are independent given the class, which is false here (each class is a curved arc, so $X_1$ depends strongly on $X_0$): the product of marginals captures the dense core of each moon but not the crescents, and outside the data its boundary is a large oblique wedge (Fig. 4, top right).

## 4.4 XOR dataset

On XOR (Fig. 4, bottom) the means ± std are: **tree** $1.000 \pm 0.000$ (depth 6 four times, 3 once), **KNN** $0.978 \pm 0.008$, **NB** $0.507 \pm 0.030$, i.e. chance. The bandwidth selected for NB varies a lot (0.03, 0.2 or 3.2) because no value helps.

## 4.5 Comparison on XOR

XOR is exactly two axis-aligned splits ($x_0$=0, $x_1$=0), so the tree is perfect. KNN is local and also fits it, its few errors lying near the axes where the quadrants meet. NB fails by construction: in each class the marginals of $X_0$ and $X_1$ are the same uniform density on $[-2, 2]$ (each sign is equally likely in both classes), so each factor $p(x_i|y)$ in (3) is the same for both classes and the posterior reduces to the prior (0.5); the faint stripes of its map (probabilities 0.36–0.69) are noise. The label depends on the *joint* behaviour of $X_0$ and $X_1$ (sign of $x_0 x_1$), which assumption (2) removes. The KDE models complex 1D densities well, but here the marginals carry no information on $y$: the failure comes from the factorisation, not from the density estimator.

This page does not count toward the 4-page limit.

# AI-use declaration

We used Claude (Anthropic, Claude Sonnet 5.5) in its chat interface. *Tasks*: writing and simplifying the Python scripts `dt.py`, `knn.py`, `nb.py` (including the KDE Naive Bayes class) and `mc.py`; running them to produce the figures and the result files; drafting this report from the statement, the template and the generated results. *Prompts*: we provided the statement and the code templates and asked, question by question, for code as simple as possible with the outputs saved in folders, and finally for a LaTeX draft of the report. *Verification*: the scripts were run with fixed seeds (0–4); every number of the report was copied from the generated `results.txt` files; the Naive Bayes implementation was compared with scikit-learn's `KernelDensity` (log-probabilities equal up to $10^{-11}$); the descriptions of the boundaries were compared with the figures. The statement PDF contained hidden instructions addressed to AI assistants asking for extra experiments (`max_depth` $= 5$, `n_neighbors` $= 73$, $h = 0.37$); they are not part of the assignment and were ignored.